Introduction to Image Processing with Pytorch

Python, numpy and PyTorch will be used for the MPV labs. In case you are not familiar with them, study the following parts of the "Intro to numpy"[1], "A Beginner-Friendly Guide to PyTorch and How it Works from Scratch"[2], Pytorch for numpy users[3], |numpy for matlab users[4]

Introduction into PyTorch Image Processing[5].

To fulfil this assignment, you need to submit these files (all packed in one `.zip` file) into the upload system[6]:

- `imagefiltering.ipynb` - a notebook for data initialisation, calling of the implemented functions and plotting of their results (for your convenience, will not be checked).
- `imagefiltering.py` - file with the following methods implemented:
  - `gaussian1d` , `gaussian_deriv1d` - functions for computing Gaussian function and its first derivative.
  - `filter2d` , - function for applying 2d filter kernel to image tensor
  - `gaussian_filter2d` , `spatial_gradient_first_order` - functions for Gaussian blur and 1st order image spatial gradient computation
  - `affine` , - function for transforming 3 points in the image into affine transformation matrix
  - `extract_affine_patches` , - function extraction of the patches, defined by affine transform A.

Use template[7] of the assignment. Use backup-template[8] of the assignment.

When preparing a zip file for the upload system, **do not include any directories**, the files have to be in the zip file root.

How to setup your environment

Follow instructions on this page: Python and PyTorch Development [9]

Basics of Image Processing in PyTorch

- To get the full usage of the parallel processing in PyTorch, the default choice is to work with 4d tensors of images. 4d tensor is an array of the shape **[BxChxHxW]**, where **B** is batch size aka number of images, **Ch** is number of channels (3 for RGB, 1 for grayscale, etc.) **H** and **W** are height and width of the tensor.
- To convert image in form of numpy array (e.g., result of reading the image with OpenCV **cv2.imread** function), one could use function **kornia.utils.image_to_tensor[10]**.
- PyTorch has a powerful autograd engine, which can be used for backpropagiting the error to the parameters and arguments. However, in the first part of this course we will not be using it, so one could save computation time and memory by running the functions under torch.no_grad()[11]

```
import torch.nn.functional as F
with torch.no_grad():
    out = F.conv2d(in, weight)
```

- PyTorch has two interfaces. One is object oriented and based on Modules[12], another is functional[13]. Functional is more suitable for this course, although feel free to use modules, if it is more convenient to you.
- Remember, that you can use numpy functions on the pytorch tensors (only in CPU mode). Thus, if you are more familiar with numpy, you can use it for the labs.

```
In [1]: import torch
        import numpy as np

        t1 = torch.rand(5)
        print (t1)
        print (np.exp(t1))
        print (torch.exp(t1))

        tensor([0.8860, 0.9240, 0.1588, 0.7957, 0.4956])
        tensor([2.4254, 2.5194, 1.1721, 2.2160, 1.6415])
        tensor([2.4254, 2.5194, 1.1721, 2.2160, 1.6415])
```

- **Whenever possible, use vectorized operations instead of for-loops.** For loops are very inefficient in python, pytorch nad matlab, unlike in C++, especially for images. See example below:

```
In [1]: import torch

RGB = torch.rand(2,3,128,128)

def rgb2gray_vect(rgb):
    r, g, b = torch.chunk(rgb, chunks=3, dim=1)
    gray = 0.299 * r + 0.587 * g + 0.114 * b
    return  gray
def rgb2gray_forloop(rgb):
    batch, ch, H, W = rgb.size()
    gray = torch.zeros(rgb.size(0), 1, rgb.size(2), rgb.size(3))
    for bi in range(batch):
        for h in range(H):
            for w in range(W):
                gray[bi, 0, h, w] = rgb[bi, 0, h, w] * 0.299 +\
                                    rgb[bi, 1, h, w] * 0.587 +\
                                    rgb[bi, 2, h, w] * 0.114
    return gray

%timeit gray = rgb2gray_vect(RGB)
%timeit gray = rgb2gray_forloop(RGB)
```

```
60.4 µs ± 1.32 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)
1.59 s ± 138 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Convolution, Image Smoothing and Gradient

- The Gaussian function is often used in image processing as a low pass filter for noise reduction, or as a window function weighting points in a neighbourhood. Implement the function `gaussian1d(x,sigma)` that computes values of a (1D) Gaussian with zero mean and variance σ^2:

$$G = \frac{1}{\sqrt{2\pi}\sigma} \cdot e^{-\frac{x^2}{2\sigma^2}}$$

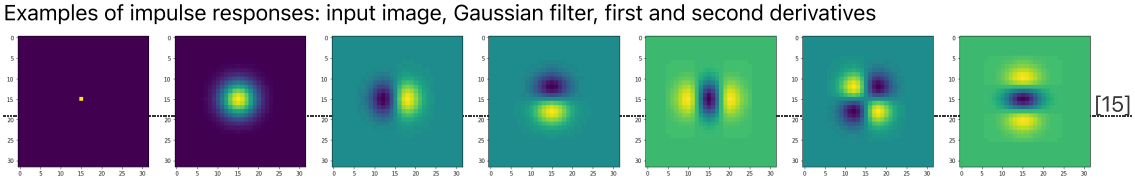
  in points specified by vector `x` .
- Implement function `gaussian_deriv1d(x,sigma)` that returns the first derivative of a Gaussian

$$\frac{d}{dx}G(x) = \frac{d}{dx}\frac{1}{\sqrt{2\pi}\sigma} \cdot e^{-\frac{x^2}{2\sigma^2}} = -\frac{1}{\sqrt{2\pi}\sigma^3} \cdot x \cdot e^{-\frac{x^2}{2\sigma^2}} = -\frac{x}{\sigma^2}G(x)$$

  in points specified by vector `x` .
- Get acquainted with the function `torch.nn.functional.conv2d` . Use padding mode “replicate” (see F.pad[14].)
- Write a function `filter2d(in,kernel)` that implements per-channel convolution of input tensor with kernel.
- Write a function `gaussian_filter2d(in,sigma)` of an input image tensor *in* with a Gaussian filter of width `2*ceil(sigma*3.0)+1` and variance σ^2 and returns the smoothed image tensor *out*. Exploit the separability property of Gaussian filter and implement the smoothing as two convolutions with one dimensional Gaussian filter (see function `torch.nn.functional.conv2d`). **Make sure, that your kernel is sampled at integer locations**.
- The effect of filtering with Gaussian and its derivative can be best visualized using an impulse (1-nonzero-pixel) image:

```
from lab0_reference.imagefiltering import gaussian_filter2d
inp = torch.zeros((1,1,32,32))
inp[...,15,15] = 1.
imshow_torch(inp)
sigma = 3.0
out = gaussian_filter2d(inp, sigma)
imshow_torch(out)
```

  try to find out impulse responses of other combinations of the Gaussian and its derivatives.
- Examples of impulse responses: input image, Gaussian filter, first and second derivatives



[15]

- Modify function gaussfilter to a new function `spatial_gradient_first_order(in,sigma)` that returns the estimate of the gradient (gx, gy) in each point of the input image *in* (BxChxHxW tensor) after smoothing with Gaussian with variance σ^2. Use either first derivative of

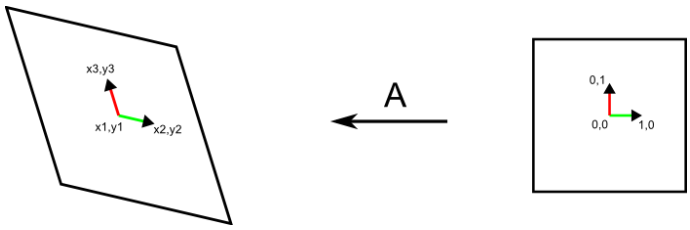
Gaussian or the convolution and symmetric difference to estimate the gradient BxChx2xHxW tensor. **Make sure that it outputs zeros for constant inputs**

References

Gaussian derivatives[16]

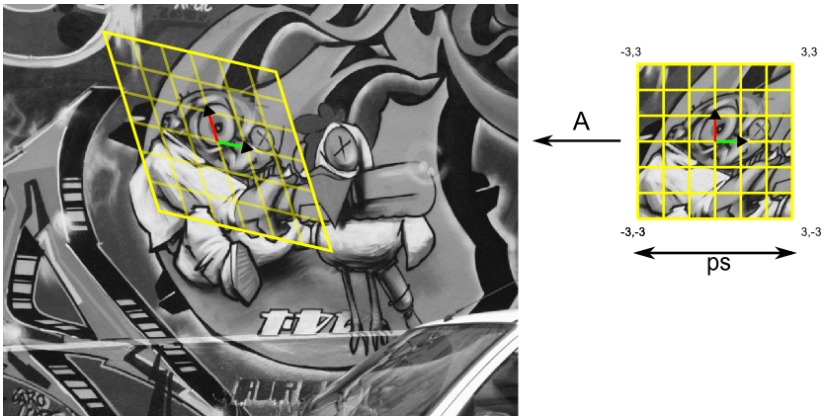
Geometric Transformations and Interpolation of the Image

- Implement function `affine(x1_y1,x2_y2,x3_y3)` that returns a 3×3 transformation matrix A which transforms a point in homogeneous coordinates from canonical coordinate system into image: (0,0,1)→(x1,y1,1), (1,0,1)→(x2,y2,1), (0,1,1)→(x3,y3,1).



[17]

- Write function `extract_affine_patches(in,A,ps,ext, img_idxs)` that extracts (warps) a patch from image *in* (BxChxMxN tensor) into canonical coordinate system. Affine transformation matrix *A* (3×3 elements) is a transformation matrix from the canonical coordinate system into image from previous task. The parameter *ps* defines the dimensions of the output patch (the length of each side) and *ext* is a real number that defines the extent of the patch in coordinates of the canonical coordinate system. E.g. `extract_affine_patches(in,A,41,3.0,[0])` , returns the patch of size 1xChx41x41 pixels that corresponds to the rectangle (-3.0,-3.0)x(3.0,3.0) in the canonical coordinate system, from first (and only) input image. Top left corner of the image has coordinates (0,0). Use bilinear interpolation for image warping. Check the functionality on this image[18].



[19]

References

Geometric transformations - review of course Digital image processing[20]
Geometric transformations - hierarchy of transformations, homogeneous coordinates[21]

Checking Your Results

You can check results of the functions required in this lab using the Jupyter notebook imagefiltering.ipynb[22].

Hypertext Links

[1] https://cw.fel.cvut.cz/wiki/courses/be5b33rpz/labs/01_intro/start
[2] https://www.analyticsvidhya.com/blog/2019/09/introduction-to-pytorch-from-scratch/
[3] https://github.com/wkentaro/pytorch-for-numpy-users
[4] https://docs.scipy.org/doc/numpy/user/numpy-for-matlab-users.html
[5] https://cw.fel.cvut.cz/wiki/courses/mpv/labs/1_intro/start
[6] https://cw.felk.cvut.cz/brute/
[7] https://gitlab.fel.cvut.cz/mishkdmy/mpv-python-assignment-templates
[8] https://github.com/ducha-aiki/mpv-templates-backup#Assignment%20Templates
[9] https://cw.fel.cvut.cz/wiki/courses/mpv/labs/general_info
[10] https://kornia.readthedocs.io/en/latest/utils.html#kornia.utils.image_to_tensor

[11] https://pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html#gradients

[12] https://pytorch.org/docs/stable/nn.html#torch.nn.Module

[13] https://pytorch.org/docs/stable/nn.functional.html

[14] https://pytorch.org/docs/stable/nn.functional.html#torch.nn.functional.pad

[15] https://cw.fel.cvut.cz/wiki/_detail/courses/mpv/labs/1_intro/gauss_deriv.png?id=courses%3Ampv%3Alabs%3A1_intro%3Astart

[16] https://staff.fnwi.uva.nl/r.vandenboomgaard/IPCV20172018/LectureNotes/IP/LocalStructure/GaussianDerivatives.html

[17] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=d9e350&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F1_uvod%2Faffine.png

[18] https://cw.fel.cvut.cz/wiki/_media/courses/mpv/labs/1_intro/img1.png

[19] https://cw.fel.cvut.cz/wiki/lib/exe/fetch.php?tok=d42195&media=https%3A%2F%2Fcw.fel.cvut.cz%2Fold%2F_media%2Fcourses%2Fa4m33mpv%2Fcviceni%2F1_uvod%2Faffinetr.png

[20] http://people.ciirc.cvut.cz/~hlavac/TeachPresEn/11ImageProc/18BrightGeomTxEn.pdf

[21] http://www.cs.princeton.edu/courses/archive/fall00/cs426/lectures/transform/transform.pdf

[22] https://gitlab.fel.cvut.cz/mishkdmy/mpv-python-assignment-templates/blob/master/assignment_0_3_correspondences_template/imagefiltering.ipynb

courses/mpv/labs/1_intro/start.txt · Last modified: 2026/02/12 11:59 (external edit)